

Programmation fonctionnelle 2

Preuves et programmes

CM11

Pierre Rousselin

Université Sorbonne Paris Nord

avril 2026

Retour sur `apply`

Retour sur `rewrite`

les prédicats `sorted` et `Permutation`

Application d'une fonction

- ▶ Rappel : pas de différence en Rocq entre
 - ▶ une fonction $h : A \rightarrow B$ et
 - ▶ une preuve h d'une implication $A \rightarrow B$.
- ▶ Appliquer une fonction :

$$\frac{\Gamma \vdash (x : A) \quad \Gamma \vdash (f : A \rightarrow B)}{\Gamma \vdash (f \ x : B)} \text{App}$$

- ▶ Utiliser une implication :

$$\frac{\Gamma \vdash A \quad \Gamma \vdash A \rightarrow B}{\Gamma \vdash B} \rightarrow\text{-E}$$

apply vers l'arrière

- ▶ **Lemma** `apply_arrière1` (`A B : Prop`) : `A -> (A -> B) -> B`.

Proof.

```
  intros h1 h2. (* but : B *)
```

```
  apply h2. (* but : A *)
```

```
  exact h1.
```

```
  Show Proof.
```

```
  (* (fun (A B : Prop) (h1 : A) (h2 : A -> B) => h2 h1) *)
```

Qed.

- ▶ Correspond à la preuve :

Lemma `terme_apply_arrière1` (`A B : Prop`) : `A -> (A -> B) -> B`.

Proof.

```
  exact (fun h1 h2 => (h2 h1)).
```

Qed.

apply vers l'avant

- ▶ **Lemma** `apply_avant1` (`A B : Prop`) : `A -> (A -> B) -> B`.

Proof.

```
intros h1 h2. (* but : B *)
apply h2 in h1. (* h1 : B *)
exact h1.
```

Qed.

- ▶ Correspond à la preuve :

Lemma `terme_apply_avant1` (`A B : Prop`) : `A -> (A -> B) -> B`.

Proof.

```
exact (fun h1 h2 => let h1 := (h2 h1) in h1).
```

Qed.

- ▶ Attention, un `apply` est un choix, qui en général affaiblit le contexte.
- ▶ Version conservative :

Lemma `apply_avant1_conservatif` (`A B : Prop`) : `A -> (A -> B) -> B`.

Proof.

```
intros h1 h2. (* but : B *)
apply h2 in h1 as h3. (* h3 : B, h1 et h2 inchangés *)
exact h3.
```

Qed.

Intermède : un lemme de simplification

► **Lemma** `add_eq_reg_l (n m p : nat) : n + m = n + p -> m = p.`

Intermède : un lemme de simplification

► `Lemma add_eq_reg_l (n m p : nat) : n + m = n + p -> m = p.`

► `Proof.`

`induction n as [| q IH].`

`- simpl; intros H; exact H.`

`- simpl; intros H; injection H.`

`apply IH.`

`exact H.`

`Qed.`

Instancier une proposition universellement quantifiée

- ▶ `add_eq_reg_l` est universellement quantifié :
`add_eq_reg_l : forall n m p : nat, n + m = n + p -> m = p`
- ▶ **Lemma** `non_unif_exple` (`a b : nat`) : `3 + a = 3 + b -> a = b`.
Proof.
 intros H.
 Check (`add_eq_reg_l 3 a b`).
 (*`(* add_eq_reg_l 3 a b : 3 + a = 3 + b -> a = b *)`*)
 apply (`add_eq_reg_l 3 a b`).
 exact H.
Qed.
- ▶ On peut donc toujours donner nous-mêmes les arguments d'un lemme pour l'appliquer (ou l'utiliser comme règle de réécriture).

Unification : remplir les trous

- ▶ On peut aussi demander à Rocq de « deviner » les arguments à donner à `add_eq_reg_1` :

```
Lemma fail_unif_exple (a b : nat) : 3 + a = 3 + b -> a = b.
```

```
Proof.
```

```
  intros H.
```

```
  Fail apply add_eq_reg_1.
```

```
  (* The command has indeed failed with message: *)
```

```
  (* Unable to find an instance for the variable n. *)
```

- ▶ Explication :
 - ▶ On demande à Rocq d'appliquer `add_eq_reg_1` au but courant.
 - ▶ Rocq va donc chercher à unifier la conclusion de `add_eq_reg_1` avec le but, donc unifier :
 - ▶ $?m = ?p$ avec
 - ▶ $a = b$.
 - ▶ Nécessairement, $?m \leftarrow a$ et $?p \leftarrow b$
 - ▶ mais rien dans le but, ne permet de deviner $?n$.

Instancier partiellement

Première solution, fournir justement l'argument manquant :

- Donner le premier argument et laisser Rocq deviner les autres :

```
Lemma unif_exple (a b : nat) : 3 + a = 3 + b -> a = b.
```

```
Proof.
```

```
  intros H.
```

```
  apply (add_eq_reg_l 3).
```

```
  exact H.
```

```
Qed.
```

- Ou marquer avec des trous (`_`), les arguments à unifier :

```
Lemma unif_exple' (a b : nat) : 3 + a = 3 + b -> a = b.
```

```
Proof.
```

```
  intros H.
```

```
  apply (add_eq_reg_l 3 _ _).
```

```
  exact H.
```

```
Qed.
```

Appliquer là où l'unification se passe mieux

Deuxième solution : appliquer à une hypothèse qui fournit davantage de contexte :

- ▶ `Lemma` `unif_exple_avant` `(a b : nat) : 3 + a = 3 + b -> a = b.`
`Proof.`
 `intros H.`
 `apply` `add_eq_reg_l` `in` `H.`
 `exact H.`
`Qed.`
- ▶ Explication : on cherche à unifier cette fois **la prémisse** de l'implication avec le type de `H`, donc
 - ▶ `?n + ?m = ?n + ?p` avec
 - ▶ `3 + a = 3 + b`
- ▶ Nécessairement :
 - ▶ `?n <- 3`
 - ▶ `?m <- a`
 - ▶ `?n <- 3` (ouf!)
 - ▶ `?p <- b`

Transitivité

- Un des cas où il faut fréquemment aider Rocq en fournissant des arguments est l'utilisation de la transitivité.

- `Check le_trans.`

```
Lemma use_trans (a b c : nat) :  
  a <= b * b -> b * b <= c -> a <= c.
```

```
Proof.
```

```
  intros H1 H2.
```

```
  Fail apply le_trans.
```

```
Abort.
```

Quelle variable est à fournir ?

Transitivité

- Un des cas où il faut fréquemment aider Rocq en fournissant des arguments est l'utilisation de la transitivité.

- `Check le_trans.`

```
Lemma use_trans (a b c : nat) :  
  a <= b * b -> b * b <= c -> a <= c.
```

```
Proof.
```

```
  intros H1 H2.
```

```
  Fail apply le_trans.
```

```
Abort.
```

Quelle variable est à fournir ?

- `Lemma use_trans (a b c : nat) :`

```
  a <= b * b -> b * b <= c -> a <= c.
```

```
Proof.
```

```
  intros H1 H2.
```

```
  apply (le_trans _ (b * b)); assumption.
```

```
Qed.
```

Retour sur `apply`

Retour sur `rewrite`

les prédicats `sorted` et `Permutation`

Rappels sur `rewrite`

- ▶ On utilise `rewrite` avec **une preuve d'égalité**.
- ▶ La tactique `rewrite` cherche alors (là où on veut réécrire)
 - ▶ un terme qui correspond au membre gauche de l'égalité
 - ▶ ou au membre droit si on a utilisé le signe `-` avant la règle
- ▶ `Check` `add_comm`. *($\ast : \text{forall } n\ m : \text{nat}, n + m = m + n$)*
- ▶ `Check` `add_assoc`. *($\ast : \text{forall } n\ m\ p : \text{nat}, n + (m + p) = n + m + p$)*
- ▶ `Lemma` `plop` `(a b c : nat) : (a + b * b) + c = b * b + (c + a)`.

Rappels sur `rewrite`

- ▶ On utilise `rewrite` avec **une preuve d'égalité**.
- ▶ La tactique `rewrite` cherche alors (là où on veut réécrire)
 - ▶ un terme qui correspond au membre gauche de l'égalité
 - ▶ ou au membre droit si on a utilisé le signe `-` avant la règle
- ▶ `Check add_comm. (* : forall n m : nat, n + m = m + n *)`
`Check add_assoc. (* : forall n m p : nat, n + (m + p) = n + m + p *)`
`Lemma plop (a b c : nat) : (a + b * b) + c = b * b + (c + a).`
- ▶ `Proof.`
 - `rewrite add_comm. (* change (a + (b * b) + c) en (c + (a + b * b))`
 - `rewrite (add_comm _ (a + _)).`
 - `rewrite add_assoc. (* une seule possibilité *)`
 - `rewrite (add_comm _ a). (* idem *)`
 - `rewrite add_assoc. (* idem *)`
 - `reflexivity.`
- `Qed.`

Subtilité : l'égalité est au bout d'une flèche

- ▶ Lorsqu'on **rewrite** avec une implication dont la conclusion est une égalité (sinon pas de **rewrite**),
- ▶ la tactique fait le même travail pour chercher quel terme correspond au membre gauche (ou au membre droit si -)
- ▶ et ensuite, demande de prouver la prémisse de l'implication, donc génère un sous-but supplémentaire.

Retour sur apply

Retour sur rewrite

les prédicats `sorted` et `Permutation`

sorted

Prédicat inductif à 3 constructeurs :

```
Inductive sorted : list nat -> Prop :=  
  | nil_sorted : sorted []  
  | one_sorted (x : nat) : sorted [x]  
  | cons_sorted (h1 h2 : nat) (t : list nat) :  
      sorted (h2 :: t) -> h1 <= h2 -> sorted (h1 :: h2 :: t).
```

Exemple

```
Lemma sorted_exple1 : sorted [2; 42; 93].
```

```
Proof.
```

```
  apply cons_sorted.
```

```
  - apply cons_sorted.
```

```
    + exact (one_sorted _).
```

```
    + lia.
```

```
  - lia.
```

```
Qed.
```

```
(* idem avec évacuation rapide des preuves faciles *)
```

```
Lemma sorted_exple1' : sorted [2; 42; 93].
```

```
Proof.
```

```
  apply cons_sorted; [| lia].
```

```
  apply cons_sorted; [| lia].
```

```
  exact (one_sorted _).
```

```
Qed.
```

Permutation

Prédicat inductif à 4 constructeurs :

```
Inductive Permutation {A : Type} : list A -> list A -> Prop :=  
  | perm_nil : Permutation [] []  
  | perm_skip (x : A) (l l' : list A) :  
    Permutation l l' -> Permutation (x :: l) (x :: l')  
  | perm_swap (x y : A) (l : list A) :  
    Permutation (y :: x :: l) (x :: y :: l)  
  | perm_trans (l l' l'' : list A) :  
    Permutation l l' -> Permutation l' l'' -> Permutation l l''.
```

Example

Lemma plop' : Permutation [3; 93; 42; 2; 1] [3; 42; 93; 1; 2].

Proof.

apply perm_skip.

apply (perm_trans _ [42; 93; 2; 1]).

 - **exact** (perm_swap _ _ _).

 - **apply** perm_skip, perm_skip.

exact (perm_swap _ _ []).

Qed.

Preuve de symétrie

```
Lemma Permutation_symm {A : Type} (l l' : list A) :  
  Permutation l l' -> Permutation l' l.
```

```
Proof.
```

```
  intros H.
```

```
  induction H as [| x l1 l2 H IH | x y t |].
```

```
  - exact perm_nil.
```

```
  - apply perm_skip. exact IH.
```

```
  - apply perm_swap.
```

```
  - apply (perm_trans _ l'); assumption.
```

```
Qed.
```